Classes aninhadas (*nested classes*)

Bib: https://docs.oracle.com/javase/tutorial/java/javaOO/nested.html

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC



Resumo

- Classes aninhadas
- Classes aninhadas estáticas e não estáticas
- Classes locais e anónimas

Resumo

- Classes aninhadas

- Classes aninhadas estáticas e não estáticas

- Classes locais e anónimas

O que é?

- Uma classe aninhada corresponde a definir uma classe dentro de outra classe (classe externa), portanto, é um membro da classe externa
- Duas categorias:
  - Classe aninhada não estática: denominada por classe interna (*inner class*)
  - Classe aninhada estática: denominada por classe aninhada estática (*static nested class*)

```
class OuterClass {
    ...
    class InnerClass { ... }
    static class NestedClass { ... }
}
```

- A classe `InnerClass` tem acesso a todos os membros de `OuterClass`, independentemente da visibilidade dos membros
- A classe `NestedClass` só tem acesso a membros estáticos de `OuterClass`
- Uma classe aninhada pode ser definida com qualquer visibilidade

Motivações para utilização de classes aninhadas

- É uma forma de agrupar classes que estão relacionadas num único sítio

- Melhora o encapsulamento: por exemplo, as classe A e B são duas classe de topo; a classe B precisa aceder aos campos da classe A que seriam privados, não fosse B; se B for definida dentro de A, consegue aceder aos campos de A e os campo de A já podem ser privados; inclusive, B pode ficar escondida dentro de A

- Pode melhorar a leitura do código fonte e melhorar a manutenção: as classes aninhadas ficam mais perto de onde são usadas

Resumo

- Classes aninhadas

- Classes aninhadas estáticas e não estáticas

- Classes locais e anónimas

Classe interna

- Consegue aceder diretamente aos membros de instância da classe externa
- A criação de uma instância da classe interna tem de ser feita no âmbito de uma instância da classe externa

```
public class OuterClass {
   String outerField = "Outer field";
   static String staticOuterField =
      "Static outer field";
   class InnerClass {
      void accessMembers() {
         System.out.println(outerField);
         System.out.println(staticOuterField);
} } }
```

Acede a campos de instância de OuterClass

e a campos estáticos

```
public class InnerDemo {
   public static void main(String[] args) {
      System.out.println("Inner class:");
      System.out.println("------------");
      OuterClass outerObject = new OuterClass();
      OuterClass.InnerClass innerObject =
         outerObject.new InnerClass();
      innerObject.accessMembers();
} }
```

Instanciar um objeto da classe interna por via de um objeto da classe externa

Classe aninhada estática

- Não consegue aceder diretamente aos membros de instância da classe externa
- Comportamento equivalente a uma classe de topo (slide a seguir)

```
public class OuterClass {
   String outerField = "Outer field";
   static String staticOuterField =
      "Static outer field";
   static class StaticNestedClass {
      void accessMembers(OuterClass outer) {
         // Compiler error: Cannot make a static
         // reference to the non-static outerField
         // System.out.println(outerField);
         System.out.println(outer.outerField);
         System.out.println(staticOuterField);
} } }
```

Acede apenas a campos estáticos

```
public class NestedStaticDemo {
   public static void main(String[] args) {
      System.out.println("\nStatic nested class:");
      System.out.println("--------------------");
      OuterClass outerObject = new OuterClass();
      OuterClass.StaticNestedClass staticNestedObject =
         new OuterClass.StaticNestedClass();
      staticNestedObject.accessMembers(outerObject);
} }
```

Instanciar um objeto da classe aninhada não exige um objeto da classe externa

Classe de topo

- Não consegue aceder diretamente aos membros de instância da classe externa

```
public class TopLevelClass {
   void accessMembers(OuterClass outer) {
      // Compiler error: Cannot make a static
      // reference to the non-static field
      // OuterClass.outerField
      // System.out.println(OuterClass.outerField);
      System.out.println(outer.outerField);
      System.out.println(
         OuterClass.staticOuterField);
} }
```

```
public class NestedStaticDemo {
   public static void main(String[] args) {
      System.out.println("\nTop-level class:");
      System.out.println("--------------------");
      TopLevelClass topLevelObject = new TopLevelClass();
      topLevelObject.accessMembers(outerObject);
} }
```

Sombreamento de nomes

- Como é resolvida a utilização de nomes iguais nos membros da classe externa e da classe interna?

```
public class ShadowDemo {
   public int x = 0;
   class FirstLevel {
      public int x = 1;
      void methodInFirstLevel(int x) {
         System.out.println("x = " + x);
         System.out.println("this.x = " + this.x);
         System.out.println(
            "ShadowDemo.this.x = " +
            ShadowDemo.this.x);
   } }
```

```
public static void main(String... args) {
      ShadowDemo st = new ShadowDemo();
      ShadowDemo.FirstLevel fl =
         st.new FirstLevel();
      fl.methodInFirstLevel(23);
   }
}
```

```
x = 23
this.x = 1
ShadowDemo.this.x = 0
```

Resumo

- Classes aninhadas
- Classes aninhadas estáticas e não estáticas
- Classes locais e anónimas

Classes locais e classes anónimas

- Uma classe aninhada pode igualmente ser definida dentro de um método
  - Apenas se pode criar instâncias da classe dentro do método onde está definida
  - Não pode ser marcada como estática
  - Consegue aceder aos membros (estáticos ou de instância) da classe externa
  - Consegue aceder às variáveis locais do método (desde que marcadas como `final` ou *efetivamente finais*). *Efetivamente final* significa que não é alterada depois de iniciada

- Uma classe aninhada pode igualmente ser definida dentro de um método e sem nome (diz-se, classe *anónima*)
  - Corresponde à implementação de um objeto do tipo interface ou à implementação de uma classe (tipicamente abstrata).
  - A definição corresponde a uma expressão (`new interface() {...}` ou `new class(args) {...}`): pode ser definida no corpo do método, como argumento na chamada a outro método, ou no retorno do método
  - Na criação do objeto fornece-se a definição dos métodos da interface ou da classe
  - Tem as mesmas características de uma classe local
  - Não suporta a definição de construtores

Classe local

```
public class LocalClassDemo {
   static String regularExpression = "[^0-9]"; // regular expression: any character except 0 to 9
   int intField = 10;
   public double m1( double d1, int i1 ) {
      final double factor1 = 0.3; double factor2 = 0.7; double res = 0;
      class LocalClass {
         private int localField;
         public LocalClass( int v ) { localField = v; }
         public double process( int v ) {
            return v + localField + d1 + factor1 + factor2 + intField + regularExpression.length();
      } }
      LocalClass local = new LocalClass( 5 );
      res = local.process( i1 );
      return res + regularExpression.indexOf('^');
} }
```

Acesso a variáveis locais/parâmetros do método. Precisam ser finais (`factor1`) ou efetivamente finais (`d1` e `factor2`)

Acesso a membros da classe externa. Acesso limitado a membros estáticos se o método `m1` for estático

Classe anónima

- Uma classe anónima é definida no contexto de uma expressão

```
public class AnonymousClassDemo {
   interface HelloWorld { public void greet();  public void greetSomeone(String someone); }
   public void sayHello() {
      HelloWorld frenchGreeting = new HelloWorld() {
         String name = "tout le monde";
         public void greet() { greetSomeone("tout le monde"); }
         public void greetSomeone(String someone) {
            name = someone;
            System.out.println("Salut " + name);
      } };
      frenchGreeting.greetSomeone("Fred");
   }
   public static void main(String... args) {
      AnonymousClassDemo myApp = new AnonymousClassDemo();  myApp.sayHello();
} }
```

No caso de implementar uma classe, usa os argumentos adequados para executar o construtor pretendido

Definida como uma expressão

Expressões *lambda* – visão geral

- Forma mais simples e sucinta de expressar a implementação de uma classe anónima
  - Tem como requisito implementar uma interface com um único método virtual

- Usado tipicamente para passar comportamento (código a ser executado) a outro método
  - Por exemplo, passar as ações a realizar sobre cada elemento de uma coleção

Quando usar classes aninhadas, classes locais, classes anónimas ou expressões *lambda*?

- Todas as variantes têm como objetivo comum agrupar no mesmo módulo classes relacionadas, melhorando o encapsulamento, legibilidade e manutenção do código
- Classes locais:
  - Quando se pretende introduzir um novo tipo classe apenas dentro do método
  - Quando se cria mais do que uma instância da classe dentro do método
  - Quando se pretende aceder a construtores da classe
- Classes anónimas:
  - Quando se cria uma única instância da interface ou da classe
  - Quando se pretende adicionar membros à classe que está a ser definida pela classe anónima
- Expressões *lambda*:
  - Quando se define o comportamento a passar a outro método correspondente a uma interface com um único método e não se pretende adicionar membros nem aceder a construtores
- Classe aninhadas:
  - Usar quando a classe aninhada é usada em mais do que um membro da classe externa
  - Estática se não precisar aceder a membros de instância da classe externa
  - Não estática se precisar aceder a membros de instância da classe externa